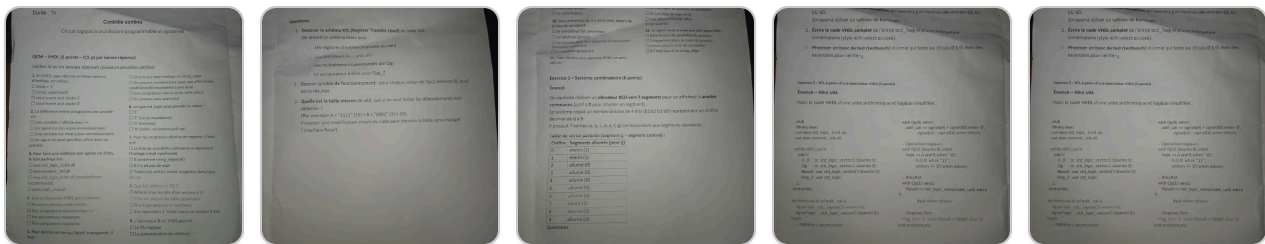


Résolution de l'épreuve



resolution de l'épreuve

Orchestrated comprehensive exam solutions across VHDL, digital logic, and architecture design

- ✗ ~~clock == 1~~
- ✓ rising_edge(clock)
- ✓ clock'event and clock='1'
- ✗ ~~clock'event and clock='0'~~ — c'est un front descendant

2. Signal vs Variable

- ✗ Une variable s'affecte avec <= — faux, c'est := pour les variables
- ✗ Un signal est mis à jour immédiatement — faux, il est mis à jour à la fin du process
- ✓ Une variable est mise à jour immédiatement
- ✗ Un signal ne peut pas être utilisé dans un process — faux, les signaux sont lisibles et assignables dans un process

3. Package addition non signée 8 bits

- ✗ ieee.std_logic_1164.all — pas de fonctions arithmétiques
- ✓ ieee.numeric_std.all — standard recommandé IEEE
- ~ ieee.std_logic_arith.all — fonctionne mais non standard
- ✗ ieee.math_real.all — pour les réels uniquement

4. Contenu d'une architecture VHDL

- ✓ Plusieurs process concurrents
- ✓ Des assignations concurrentes <=
- ✗ Un seul process maximum — faux, on peut en avoir plusieurs
- ✓ Des component instanciés

5. Décrire un verrou (latch) transparent

- ✗ Un process avec horloge et rising_edge — c'est un registre, pas un latch
- ✓ Un process combinatoire avec une affectation conditionnelle incomplète (sans else)
- ✗ Une assignation concurrente with select — ne génère pas de latch
- ✗ Un process avec wait until — génère un registre avec attente

6. Valeurs de std_logic

- ✓ '1' — valeur logique haute

- ✓ 'Z' — haute impédance (trois états)
- ✓ 'X' — valeur inconnue/conflit
- ✓ 'H' — faible, résistance pull-up

std_logic possède 9 valeurs : U, X, 0, 1, Z, W, L, H, -

7. Process décrivant un registre

- ~ La liste de sensibilité contient uniquement l'horloge — vrai pour reset synchrone seulement
- ✓ Il contienne rising_edge(clk)
- ✓ Il n'y ait pas de wait (avec liste de sensibilité)
- ✗ ~~Toutes les sorties soient assignées dans tous les cas — obligatoire pour le combinatoire, pas pour le registre~~

8. Que fait (others => '0') ?

- ✓ Affecte tous les bits d'un vecteur à '0'
- ✗ ~~Crée un vecteur de taille générique — non, la taille est inférée du contexte~~
- ✗ ~~N'est pas autorisé en synthèse — faux, c'est très utilisé~~
- ✓ Est équivalent à "0000" pour un vecteur 4 bits

9. Opérateur & en VHDL

- ✗ ~~Le OU logique — c'est l'opérateur or~~
- ✓ La concaténation de vecteurs — ex: "10" & "11" = "1011"

10. process(a, b) si c absent de la liste

- ✗ ~~Le simulateur fait une erreur — non, juste un warning éventuel~~
- ✗ ~~Un latch est généré — non, c'est lié à des affectations incomplètes~~
- ✓ Le résultat peut être incorrect en simulation (mauvaise réactivité)
- ✓ La synthèse ignorera c — en synthèse, le circuit réagit à tous les signaux lus

11. Mémoire ROM en VHDL

- ✗ ~~L'addition entière~~
- ✗ ~~La comparaison~~

- ✓ Un case dans un process combinatoire
- ✓ Un constant de type array
- ✗ Une affectation avec after — pour la simulation uniquement
- ✗ Un wait for — simulation uniquement, non synthétisable

12. Reset asynchrone

- ✓ Dans la liste de sensibilité du process — obligatoire pour réagir immédiatement
- ✗ Uniquement dans le corps du process — insuffisant sans la liste
- ✗ Jamais dans la liste de sensibilité — faux
- ✗ À l'intérieur d'un rising_edge — faux, le reset asynchrone est en dehors du if rising_edge

Exercice 1 — Décodeur BCD vers 7 segments

Q1 — Équation logique simplifiée de g (tableau de Karnaugh)

$g=1$ (éteint) pour 0,1,7 — $g=0$ (allumé) pour 2,3,4,5,6,8,9. Cases 10–15 = don't care (X)

$b_3b_2 \setminus b_1b_0$	00	01	11	10
00	1	1	0	0
01	0	0	1	0
11	X	X	X	X
10	0	0	X	X

Groupe vert : {0000, 0001} → $b_3' \cdot b_2' \cdot b_1'$

Groupe violet : {0111, 1111(X)} → $b_2 \cdot b_1 \cdot b_0$

$$g = b_3' \cdot b_2' \cdot b_1' + b_2 \cdot b_1 \cdot b_0$$

Q2 — Code VHDL complet (style with-select)

```
library ieee;
use ieee.std_logic_1164.all;

entity bcd_7seg is
  port (
    b  : in  std_logic_vector(3 downto 0);
    seg : out std_logic_vector(6 downto 0) -- a,b,c,d,e,f,g
  );
end entity bcd_7seg;
```

```

architecture combinatoire of bcd_7seg is
begin
    -- Encodage anode commune (0=allumé, 1=éteint)
    -- seg = "abcdefg"
    with b select
        seg <= "0000001" when "0000", -- 0 : g éteint
              "1001111" when "0001", -- 1
              "0010010" when "0010", -- 2
              "0000110" when "0011", -- 3
              "1001100" when "0100", -- 4
              "0100100" when "0101", -- 5
              "0100000" when "0110", -- 6
              "0001111" when "0111", -- 7 : g éteint
              "0000000" when "1000", -- 8
              "0000100" when "1001", -- 9
              "1111111" when others; -- invalide
end architecture combinatoire;

```

Q3 — Testbench avec assertions sur g

```

library ieee;
use ieee.std_logic_1164.all;

entity tb_bcd_7seg is
end entity tb_bcd_7seg;

architecture sim of tb_bcd_7seg is
    component bcd_7seg is
        port (b : in std_logic_vector(3 downto 0);
              seg : out std_logic_vector(6 downto 0));
    end component;
    signal b : std_logic_vector(3 downto 0) := (others => '0');
    signal seg : std_logic_vector(6 downto 0);
begin
    UUT : bcd_7seg port map(b => b, seg => seg);

    process
    begin
        -- Cas 0 : g doit être éteint (= '1')
        b <= "0000"; wait for 10 ns;
        assert seg(0) = '1'
            report "ERREUR chiffre 0 : g incorrect" severity error;

        -- Cas 1 : g éteint
        b <= "0001"; wait for 10 ns;
        assert seg(0) = '1'
            report "ERREUR chiffre 1 : g incorrect" severity error;
    end process;
end architecture sim;

```

```

-- Cas 2 : g allumé (= '0')
b <= "0010"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 2 : g incorrect" severity error;

-- Cas 3 : g allumé
b <= "0011"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 3 : g incorrect" severity error;

-- Cas 4 : g allumé
b <= "0100"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 4 : g incorrect" severity error;

-- Cas 5 : g allumé
b <= "0101"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 5 : g incorrect" severity error;

-- Cas 6 : g allumé
b <= "0110"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 6 : g incorrect" severity error;

-- Cas 7 : g éteint
b <= "0111"; wait for 10 ns;
assert seg(0) = '1'
    report "ERREUR chiffre 7 : g incorrect" severity error;

-- Cas 8 : g allumé
b <= "1000"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 8 : g incorrect" severity error;

-- Cas 9 : g allumé
b <= "1001"; wait for 10 ns;
assert seg(0) = '0'
    report "ERREUR chiffre 9 : g incorrect" severity error;

report "Simulation terminée – tous les cas testés" severity note;
wait;
end process;
end architecture sim;

```



Note : seg(0) = segment g car seg = "abcdefg" donc l'indice 0 correspond à g dans le vecteur.

Exercice 3 — Mini UAL (RTL à partir de VHDL)

Q1 — Schéma RTL (description structurelle)

Blocs principaux issus du code VHDL :